

NAME : GAYATHERI S

COURSE CODE : CSA0613

REG NO: 192521306

COURSE NAME : DESIGN AND

ANALYSIS OF ALGORITHMS

CO2 - ASSESSMENT TOOL – 03

Q31. Debugging Selection Sort Code

1. Problem Analysis

The given Selection Sort program contains an error in the outer loop.

Incorrect code: for

i in range(n-2):

This executes only **n-2 passes**, whereas Selection Sort requires **n-1 passes**.

As a result, the second-last element is not checked correctly, leaving the array partially unsorted for some inputs.

Example:

Input:

[5, 4, 3, 2, 1]

Using the incorrect code may not completely sort the last two elements.

2. Corrected and Optimized Source Code

```
1
2- def selection_sort(arr):
3     n = len(arr)
4
5     # Perform n-1 passes
6-     for i in range(n - 1):
7
8         # Assume current position has minimum value
9         min_idx = i
10
11        # Find the minimum element
12-        for j in range(i + 1, n):
13-            if arr[j] < arr[min_idx]:
14-                min_idx = j
15
16        # Swap only if required
17-        if min_idx != i:
18-            arr[i], arr[min_idx] = arr[min_idx], arr[i]
19
20    return arr
21
22
23 # Driver Code
24 arr = [64, 25, 12, 22, 11]
25
26 print("Original Array:", arr)
27
28 selection_sort(arr)
29
30 print("Sorted Array:", arr)
```

3. Output

```
Output
Original Array: [64, 25, 12, 22, 11]
Sorted Array: [11, 12, 22, 25, 64]

=== Code Execution Successful ===
```

4. Debugging Analysis

Error Found

- Outer loop uses:

`range(n-2)`

Why it is wrong

- Executes one pass less.
- Remaining unsorted elements are not processed. **Correction** `range(n-1)`

This performs all required passes.

Optimization

Before swapping: if

`min_idx != i:`

This avoids unnecessary swaps when the minimum element is already in the correct position.

Benefits:

- Fewer write operations.
- Better performance in practice.
- Same sorting result.

5. Time Complexity

Case	Complexity
------	------------

Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity: $O(1)$

Algorithm Steps

1. Start from the first element.
2. Find the minimum element in the unsorted portion.
3. Swap it with the current position only if needed.
4. Repeat until the array is sorted.

Result

Input Array: [64,

25, 12, 22, 11]

Output:

Original Array: [64, 25, 12, 22, 11]

Sorted Array: [11, 12, 22, 25, 64]

Result Analysis:

- The corrected Selection Sort algorithm successfully sorts the array in ascending order.
- Changing the outer loop from $\text{range}(n-2)$ to $\text{range}(n-1)$ ensures all required passes are executed.
- The optimization if $\text{min_idx} \neq i$: avoids unnecessary swaps, making the algorithm more efficient without changing its time complexity.

Conclusion:

The corrected program produces the expected sorted output, performs complete sorting, and avoids unnecessary swaps. It has a time complexity of $O(n^2)$ and a space complexity of $O(1)$.